Phase 6: RISC-V Cache

EEL 4768 Computer Architecture: Spring 2026

Project Introduction

In this project phase, you will be building upon the previous phases of the project to add a single layer cache to your multi cycle RISC-V 32RI processor in Verilog. A top module skeleton will be provided which you ***must use*** or the autograder may not be able to process your files.

Your processor should ***only use instructions found in the provided RV32I reference card posted in webcourses***. Using any instructions outside of the base RV32I instruction set will result in a zero for the specific problem.

Submission Requirements

You are permitted to use generative artificial intelligence (e.g. ChatGPT); if you choose to do so, indicate where you use AI in your submission, and why you used it. Please ensure your usage of LLMs adheres to the course policies.

Submit the Verilog files as individual, unzipped files directly to Gradescope. In addition to the Verilog files, please include a word document or pdf detailing the contributions of you and your team members, along your use of AI.

- RISCV_TOP.v
- Additional Verilog files
- assembler.(py, cpp)
- group_contributions.pdf
- signals.yaml

In Gradescope, after submitting you can select your group for that submission. One submission is needed, but if you fail to select your group members in that submission, they may receive a zero for this phase.

Grading Rubric

Implementation		Points
Cache		
	Eviction policy	
	LRU	2
	PLRU	1
	N-mapped associativity	2
	Next-line Prefetch	1
	Write-back on Hit Write-allocate on Miss	2
Multicycle CPU integration		
	Assembler	2
	Forwarded 5-stage pipeline CPU	4
Group contribution and AI report		1
Total		***15***

.

Environment setup

You may use any simulator as long as it's capable of simulating Verilog. If you want to use Vivado like how you used it in digital systems, you are more than welcome to.

Another simulator that is recommended to be used if Verilator, which compiles the simulation of the Verilog files with a C++ wrapper. For assistance with installation, please come to my office hours.

Icarus is another lightweight option.

Your assembler can be writing in one of two languages of your choice.

- C++
- Python

Cache

The goal of this phase is to integrate a cache into our RISCV processor, sitting in between both the instruction and data memory. Each memory type will now have its own small cache, sharing a main memory for instructions and data.

Within the shared memory, instructions (.text) will inhabit the range of [0x00400000, 0x0FFFFFFF], while initialized data (.data) will inhabit the range of [0x1001000, 0x1003FFFF], and heap data will inhabit the range of [0x10040000, 0xFFFEFFFF].

When accessing data from memory, your processor should first query the cache for the data, and update, accordingly, taking 1 cycle. If the data is not in the cache, your processor then must retrieve the data from main memory, which takes a penalty of 100 cycles in addition to the cycle required to query the cache.

Your cache should be completely parameterizable (i.e., cache size, block size, associativity, etc).

Below will be detail each attribute of the cache design that will be implemented.

Eviction Policy

The eviction policy determines which cache block is removed when a new block must be loaded into a full set. Two common policies are:

Least recently used (LRU)

LRU evicts the block that has not been accessed for the longest time. It provides high accuracy by closely approximating optimal replacement behavior. However, true LRU is costly to implement in hardware, especially at higher associativity, because it requires maintaining an exact ordering of accesses within each set (e.g., via counters or ordered structures).

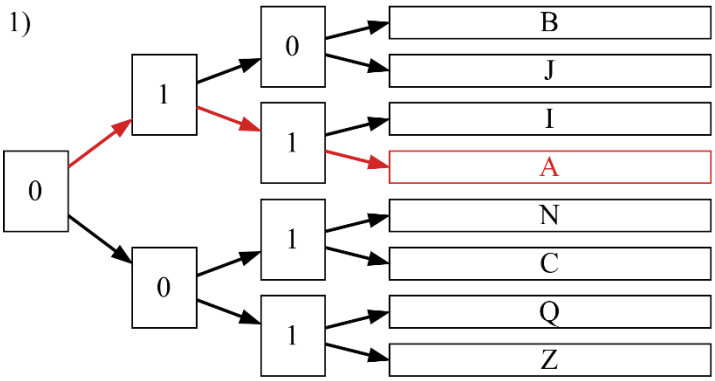
Psudo-LRU (PLRU)

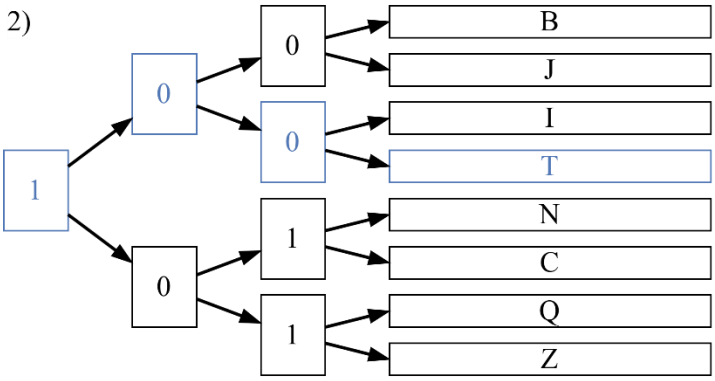
PLRU reduces the hardware overhead of true LRU by approximating recency information. A common implementation uses a small set of bits organized as a binary tree, where each access updates bits along the path to indicate which subtree was most recently used. This allows the hardware to select a victim with significantly less state and logic. While PLRU is not perfectly accurate, it performs very close to LRU in practice, making it a widely used trade-off in modern CPU cache design.

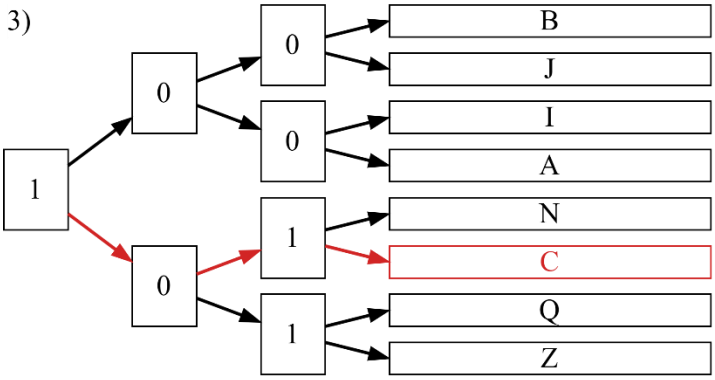
Below is an example of PLRU.

1. Current state of the PLRU points to block A to be evicted.

2. Bit values are updated and block A is replaced with block T.
3. Next state of PLRU, where block c will be evicted.







Associativity

Associativity defines how memory blocks are mapped to cache locations. Specifically, how many possible locations (ways) a given block can occupy within the cache.

Direct mapped

Each memory block maps to exactly one cache line, determined by the index bits of its address. This makes the design simple and fast, but it is prone to conflict misses since multiple blocks may compete for the same line.

Fully associative

A memory block can be placed in any cache line. There is no index field; instead, all tags are compared in parallel to find a match. This eliminates conflict misses but is expensive in hardware due to the need for associative (parallel) tag comparisons.

N-mapped associativity.

The cache is divided into multiple sets, and each set contains *N* lines (ways). A memory block maps to exactly one set (via index bits) but can occupy any of the *N* ways within that set. This design balances the trade-offs between direct-mapped and fully associative caches, reducing conflict misses while keeping hardware cost manageable. Direct mapped and full associative cache can be implemented within N-mapped associativity.

Prefetching

Cache misses are commonly categorized as **compulsory (cold) misses** and **conflict misses** (in addition to capacity misses).

- **Compulsory misses** occur the first time a block is accessed.
- **Conflict misses** occur when multiple blocks map to the same set and evict each other.

Prefetching is a technique used to reduce miss latency and frequency by loading data into the cache *before* it is explicitly requested by the processor. By bringing in data early, prefetching can significantly reduce compulsory misses and, in some cases, hide memory access latency. However, it does not eliminate conflict or capacity misses and can even increase them if it pollutes the cache with unused data.

Next-line perfecting.

A simple prefetching strategy is next-line prefetching, where, upon accessing a cache block, the hardware also fetches the immediately following block (based on spatial

locality). This approach is easy to implement and works well for sequential memory access patterns but may waste bandwidth and cache space if the prefetched data is not used.

Write Policy

The write policy defines how a cache handles write operations, both on hits and misses.

Write-back on hit

On a write hit, the cache updates only the cache line and marks it as **dirty**, indicating it has been modified. The updated data is written back to main memory only when the block is evicted. This reduces memory traffic compared to write-through but requires additional hardware (dirty bits) and more complex eviction handling.

Write-allocate on miss

On a write miss, the cache first loads the corresponding block from main memory into the cache, then performs the write in the cache. This takes advantage of spatial and temporal locality, as future accesses to the same block is likely. Write-allocate is commonly paired with write-back policies.

Assembler

We will reuse our assemblers from Phase 5, but in this phase your assembler will be required to return two files of hex values, instr.txt and data.txt. Both should store a byte per line and use little endian. For example, if your instruction or data is

0x00AD7318

Then it should save as

0x18

0x73

0xAD

0x00

The instr.txt will contain only encoded instruction commands, while the data.txt will only contain word data that is initially stored in data. The assembler should take in two arguments <file.s> and <output_directory>. The first argument should just be the input file, while the second argument is the output directory for your assembled instr.txt and data.txt files. Your assembler should end each file with an ebreak signal, which is currently in each assembly file that is an input to your assembler. The ebreak instruction should decode to 0x00100073.

Signals.yaml

Finally, your submission should include a signals.yaml file, which is attached to this assignment.

For those not familiar with yaml, it is a file that defines a data structure of key value pairs. A key, or name, will point to a value, which can be some object. In our example, they are string tuples and lists.

This file is needed by the internal simulator to find specific signals in your design. Selecting the wrong signals will make it not work.

IF_ID_Instruction:

This signal is the output register holding the instruction from your if/id stage.

IF_ID_PC:

This signal is the output register holding the PC from your if/id stage.

Instruction_Cache:

This signal points to your instruction cache register array.

Data_Cache

This signal points to your Data_Cache register array.

<u>Making a signal:</u>

If the signal exists in your RISCV_TOP.v file, then all you need to do is put its name, as shown for the IF_ID_Instruction and IF_ID_PC. If a signal exists inside a module, then you need to add the module names before the name of the signal. These need to be the name of the instantiation, not the module itself.

Examples are shown on the next page, which align with my signals.

Finally, if you want additional tracing of your input, you can add the module instantiation names to the trace list. Every signal will be traced for you to view.

Example module structure:

```
module RISCV_TOP(
    …
);

Wire wInstr, wPC;

MEM mem_stage (…);
endmodule
```

```
module MEM(
    …
);

module DATA_MEMORY data_memory(…);
endmodule
```

```
module MEM(
    …
);

reg [B-1:0] rDataMem [0:(K*4)-1]; // 4KB data memory, byte-addressable
endmodule
```